Further, since a trace is made up of a collection of spans, you would expect that when looking at a trace, you will see all the spans that make up the trace. This sounds obvious, but it can be more difficult in practice. Several challenges, with the following being the most prominent, can cause broken and partial traces:

- *Context propagation*—To see all spans, every service needs to propagate the trace information (root span and metadata). What if one of the services (you don't own) doesn't participate in distributed tracing? What if you're using a cloud provider API that is not instrumented and, hence, doesn't propagate? How about the case in which an API supports a different trace header than what you use (e.g., proprietary versus W3C trace header)? To address this, you may need to upgrade your API integration (supporting a new header type); use a middleware or translation layer to translate header types; and, potentially, lobby with your provider to support tracing.
- *Dropped spans*—This can be due to a faulty or starved agent (e.g., an OpenTelemetry collector's in-memory queue overflows and may even be OOMed—that is, the Linux kernel kills the process, due to not enough memory being available) or some intermediary being configured to filter certain spans (e.g., a Gateway deployment in OpenTelemetry; https://opentelemetry.io/docs/collector/deployment/), where collectors used at the edge send spans to a central collector, unbeknownst to you.

The investment you're making with distributed tracing comprises the generation of the spans (ideally, via auto-instrumentation); the collection (think, for example, an OpenTelemetry collector); the ingestion and storage costs; and, last but not least, the query or usage in the frontend (e.g., for the purpose of graphing traces in a waterfall diagram or to provide service maps and other aggregates).

Let us zoom in on the topic of how to select spans that are worth being collected and ingested into backends: in other words, sampling.

8.3.1 Sampling

Distributed tracing means that in production environments you regularly see large volumes of data being generated. Given its natural alignment with distributed systems (microservices), it's not uncommon to be faced with dozens of billions of spans, representing a TB of data per day. The question is then as follows: How can we select relevant spans to collect and store? The term we're using here for the selection process is *sampling*. Mind you, there are some innate limitations to sampling (https://research.swtch.com/sample), but ignoring that for now, the following high-level strategies are available:

- *No sampling*—You ingest all spans, which could be useful for dev/test environments, to evaluate a tool, or if you have a tool that doesn't charge on a volume basis. Caching can help a lot for the query path when you're investigating traces, but the span ingestion path is still challenging.